



Модуль 3. Сетевое программирование и протоколы

Введение в модуль

В Модуле 2 мы изучили, как Python управляет конкурентностью через event loop. Но event loop — это всего лишь диспетчер. Чтобы понять, **что именно** он диспетчирует, нужно разобраться в сетевых протоколах: как биты превращаются в HTTP-запросы, почему TCP гарантирует доставку, а UDP — нет, и почему HTTP/3 может быть быстрее HTTP/2.

Этот модуль — мост между теорией сетей и практикой FastAPI. Мы пойдём от физического уровня до прикладного, разберём эволюцию HTTP и поймём, почему выбор протокола напрямую влияет на пропускную способность ML-сервиса.

3.1. Модель OSI и TCP/IP

3.1.1. Что такое протокол и зачем нужны уровни

Протокол — это набор правил, определяющих формат, порядок и семантику сообщений, которыми обмениваются две стороны. В сетях стороны — это хосты (компьютеры), процессы или устройства.

Проблема: сеть — чрезвычайно сложная система. Чтобы управлять сложностью, инженеры разделили её на **уровни абстракции**. Каждый уровень решает свою задачу и предоставляет сервисы уровню выше, не раскрывая деталей реализации.

Это классический пример **инкапсуляции** в инженерии: уровень N видит только уровень $N-1$ как «чёрный ящик».

3.1.2. Модель OSI: семь уровней

Модель OSI (Open Systems Interconnection) — теоретический эталон, разработанный ISO в 1984 году.

Уровень	Название	Единица данных	Задача	Примеры
7	Прикладной (Application)	Данные	Интерфейс для приложений	HTTP, FTP, SMTP, DNS
6	Представления (Presentation)	Данные	Шифрование, кодировка, сериализация	TLS/SSL (частично), ASN.1
5	Сеансовый (Session)	Данные	Управление сессиями, диалогами	NetBIOS, RPC (устаревшие)
4	Транспортный (Transport)	Сегмент	Надёжность, порты, потоки	TCP, UDP
3	Сетевой (Network)	Пакет	Маршрутизация, адресация	IP, ICMP, OSPF
2	Канальный (Data Link)	Кадр	Передача по физической среде, MAC-адреса	Ethernet, Wi-Fi (802.11), ARP
1	Физический (Physical)	Бит	Электрические/оптические сигналы	Витая пара, оптоволокно, радио

Важно: OSI — это модель, не реализация. В реальном Интернете используется стек **TCP/IP**, который объединяет некоторые уровни.

3.1.3. Стек TCP/IP: четыре уровня

TCP/IP — практический стек, на котором построен Интернет.

TCP/IP	Соответствие OSI	Протоколы
Прикладной (Application)	5–7 OSI	HTTP, HTTPS, DNS, SSH, FTP
Транспортный (Transport)	4 OSI	TCP, UDP
Межсетевой (Internet)	3 OSI	IP, ICMP, IGMP
Канальный (Link)	1–2 OSI	Ethernet, Wi-Fi, ARP

Когда FastAPI-сервер получает HTTP-запрос, данные проходят через все уровни «снизу вверх»:

1. **Физический:** электрические импульсы по витой паре или Wi-Fi-радиоволны.
2. **Канальный:** Ethernet-кадры с MAC-адресами.
3. **Сетевой:** IP-пакеты с адресами источника и назначения.
4. **Транспортный:** TCP-сегменты с портами (например, порт 443 для HTTPS).

5. **Прикладной:** HTTP-запрос в виде текста.

3.1.4. TCP: надёжность в ненадёжном мире

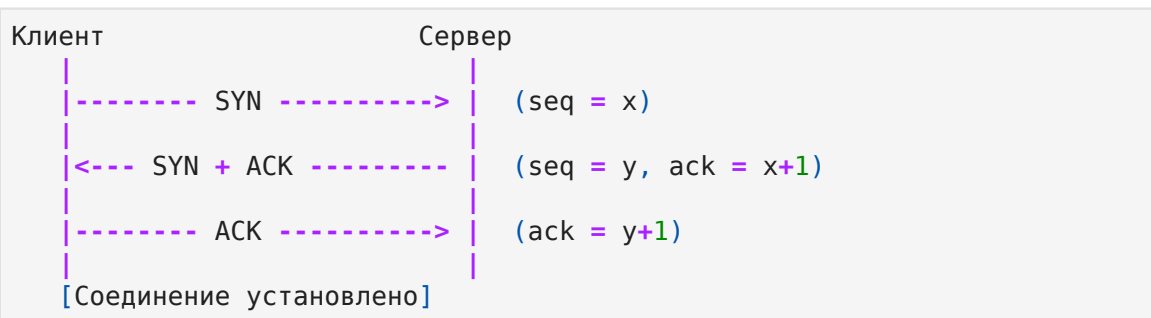
IP — протокол **без установления соединения** и **без гарантий доставки**. Пакеты могут теряться, дублироваться, приходить в неправильном порядке.

TCP (Transmission Control Protocol) решает эти проблемы, добавляя:

1. **Установление соединения** (three-way handshake).
2. **Нумерацию байтов** (sequence numbers).
3. **Подтверждения** (ACK — acknowledgements).
4. **Повторную передачу** при потере.
5. **Управление потоком** (flow control) — не усыплять медленного получателя.
6. **Управление перегрузкой** (congestion control) — не коллапсировать сеть.

Three-way handshake (установление соединения)

In []:



- **SYN** (synchronize): клиент сообщает начальный sequence number (\$x\$).
- **SYN-ACK**: сервер подтверждает (\$ack = x+1\$) и сообщает свой sequence number (\$y\$).
- **ACK**: клиент подтверждает (\$ack = y+1\$).

Только после этого начинается передача данных. Задержка — **1 RTT** (Round Trip Time).

Sequence numbers и ACK

TCP рассматривает поток данных как последовательность **байтов**, а не

сообщений. Каждый байт имеет номер.

```
In [ ]: Клиент отправляет: [seq=1000, data="Hello"]
        Сервер отвечает:  [ack=1005] (1000 + 5 байт)
```

Если ACK не приходит в течение таймаута — сегмент отправляется повторно.

Окно перегрузки (Congestion Window)

TCP не отправляет данные «вприпрыжку». Он использует алгоритм **AIMD** (Additive Increase, Multiplicative Decrease):

- **Slow Start:** окно перегрузки $cwnd$ начинается с небольшого значения (обычно 2–10 MSS) и **экспоненциально** растёт с каждым подтверждённым RTT.
- **Congestion Avoidance:** после порога $ssthresh$ $cwnd$ растёт **линейно** (+1 MSS за RTT).
- **Потеря пакета:** $cwnd$ **делится пополам** (multiplicative decrease), $ssthresh$ устанавливается в новое $cwnd$.

Это предотвращает коллапс сети, когда слишком много хостов начинают передачу одновременно.

Flow Control vs Congestion Control

	Flow Control	Congestion Control
Что ограничивает	Скорость отправителя под возможности получателя.	Скорость отправителя под пропускную способность сети.
Механизм	Окно получателя (rwnd) в заголовке TCP.	Окно перегрузки (cwnd), вычисляемое алгоритмом.
Где измеряется	Буфер получателя.	Потери и задержки в сети.

Реальное окно отправки: $\min(rwnd, cwnd)$.

3.1.5. Порты и сокеты

Порт (16-битное число, 0–65535) — это адрес **процесса** внутри хоста. IP-адрес доставляет пакет на машину, порт — конкретной программе.

Диапазон	Назначение
0-1023	Well-known (HTTP: 80, HTTPS: 443, SSH: 22)
1024-49151	Registered (PostgreSQL: 5432, Redis: 6379)
49152-65535	Dynamic/Private (временные порты клиентов)

Сокет (socket) — это абстракция «конечной точки соединения». В Python:

```
In [4]: import socket

# Создание TCP-сокета
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(("example.com", 80))
sock.send(b"GET / HTTP/1.1\r\nHost: example.com\r\n\r\n")
response = sock.recv(4096)
```

Сокет — это интерфейс между прикладным уровнем (ваш код) и транспортным (TCP в ядре ОС).

3.1.6. HTTP как текстовый протокол поверх TCP

HTTP (HyperText Transfer Protocol) — прикладной протокол, работающий поверх TCP.

```
In [ ]: Клиент                                Сервер

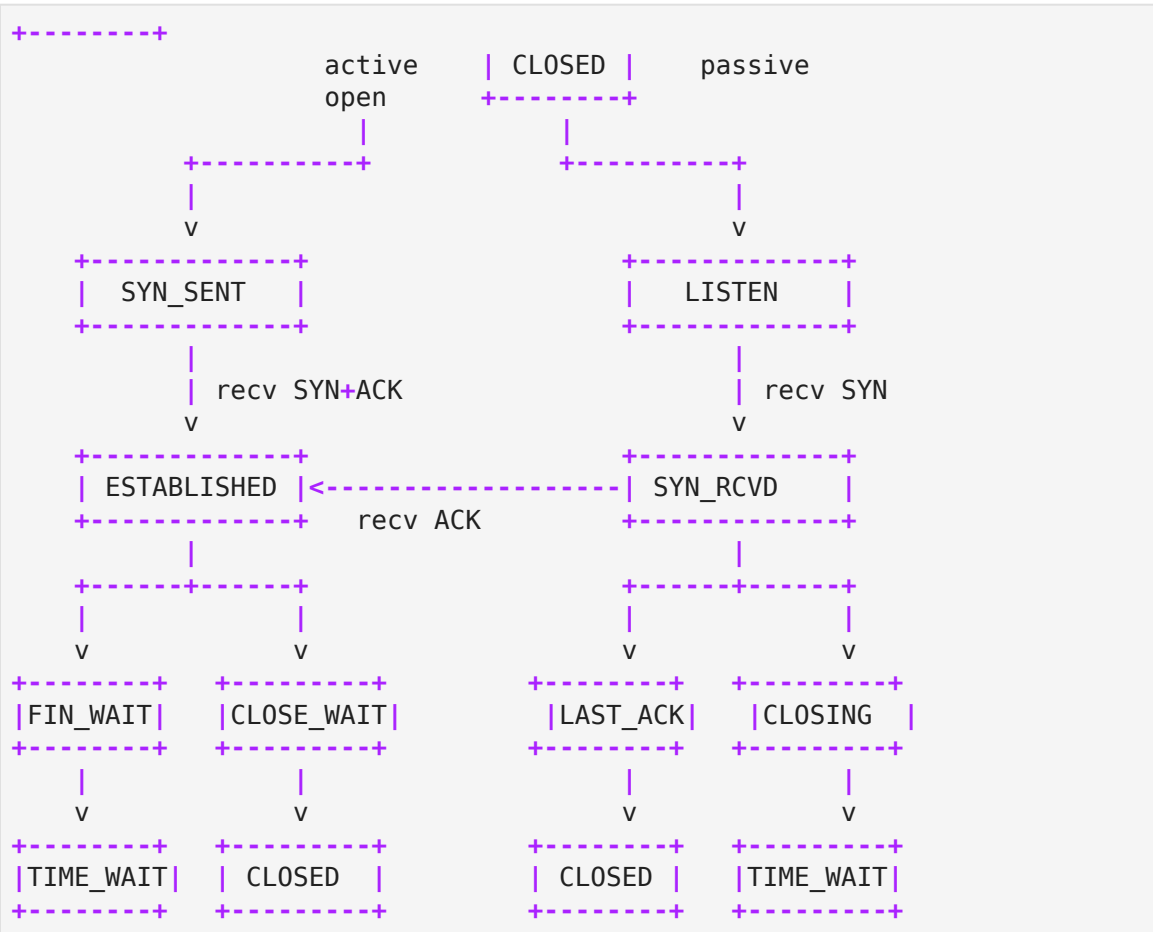
|===== TCP handshake (SYN-SYN/ACK-ACK) =====|
|----- GET /api/users HTTP/1.1 ----->|
|      Host: api.example.com               |
|      Accept: application/json             |
|<----- HTTP/1.1 200 OK -----|
|      Content-Type: application/json       |
|      Content-Length: 47                  |
|      {"users": [{"id": 1, "name": "A"}]}  |
|===== TCP close (FIN-ACK-FIN-ACK) =====|
```

HTTP не знает о TCP, а TCP не знает о HTTP. TCP предоставляет «трубу» — надёжный байтовый поток. HTTP пишет текст в эту трубу.

3.1.7. Математическая подоплека: конечные автоматы для TCP

TCP-соединение на каждой стороне описывается **конечным автоматом** состояний:

In []:



Ключевые состояния:

- **ESTABLISHED:** соединение активно, передаются данные.
- **TIME_WAIT:** после закрытия сокет держится $2 \times \text{MSL}$ (Maximum Segment Lifetime, обычно 60–120 секунд), чтобы принять «запоздалые» пакеты и корректно ответить на них RST.

AIMD как динамическая система:

Пусть $W(t)$ — размер окна перегрузки в момент t . В фазе congestion avoidance:

$$\frac{dW}{dt} = \frac{1}{RTT}$$

При потере (предположим, потери происходят с вероятностью, зависящей от W):

$$W_{\text{new}} = \frac{W_{\text{old}}}{2}$$

Это создаёт **пилообразную динамику**: линейный рост до потери, резкое падение, снова рост. В среднем пропускная способность TCP:

$$\text{Throughput} \approx \frac{1.22 \cdot \text{MSS}}{\text{RTT} \cdot \sqrt{p}}$$

где p — вероятность потери пакета. Это объясняет, почему TCP на спутниковом канале (большой RTT) медленнее, чем на оптоволокне — даже при одинаковой пропускной способности физического уровня.

3.2. HTTP/1.1, HTTP/2, HTTP/3

3.2.1. HTTP/1.1: эволюция и узкие места

HTTP/1.1 (1997) — долгоживущий стандарт. Ключевые особенности:

Persistent Connections (Keep-Alive):

В HTTP/1.0 каждый запрос требовал нового TCP-соединения (и нового handshake). HTTP/1.1 ввёл заголовок `Connection: keep-alive`, позволяющий отправлять несколько запросов в одном TCP-соединении.

```
In [ ]: GET /page1 HTTP/1.1
        Host: example.com

        GET /page2 HTTP/1.1
        Host: example.com
```

Pipelining:

Клиент может отправить несколько запросов, не дожидаясь ответа на предыдущий. Но сервер **обязан** отвечать в том же порядке.

```
In [ ]: Клиент: Req1 Req2 Req3
        Сервер:      Resp1 Resp2 Resp3
```

Head-of-Line Blocking (HoL):

Проблема pipelining: если `Req1` требует тяжёлой обработки (например, запрос к медленной БД), `Req2` и `Req3` вынуждены ждать, даже если они лёгкие. Ответы должны идти строго по порядку.

Браузеры решили эту проблему **domain sharding**: открывали до 6 TCP-соединений на один домен, чтобы запрашивать ресурсы параллельно. Но это порождало накладные расходы на handshake и congestion control для каждого соединения.

Chunked Transfer Encoding:

Позволяет серверу отправлять ответ частями, не зная `Content-Length` заранее:

```
In [ ]: HTTP/1.1 200 OK
        Transfer-Encoding: chunked

        1a\r\n
        {"status": "processing"}\r\n
        0\r\n
        \r\n
```

Это критично для streaming-ответов (например, генеративных ML-моделей, выдающих токены по одному).

3.2.2. HTTP/2: мультиплексирование и бинарность

HTTP/2 (2015, RFC 7540) — кардинальная переработка.

Бинарное фреймирование

HTTP/1.1 — текстовый протокол. HTTP/2 — **бинарный**. Данные разбиваются на **фреймы** (frames) фиксированной структуры:

```
In [ ]: +-----+
        |                                     |
        |               Length (24)           |
        +-----+-----+-----+-----+
        | Type (8)   | Flags (8)   |
        +-----+-----+-----+-----+
        |R|               Stream Identifier (31) |
        +-----+-----+-----+-----+
        |               Payload (variable)      |
        +-----+-----+-----+-----+
```

Типы фреймов:

- **HEADERS**: заголовки запроса/ответа.
- **DATA**: тело сообщения.
- **SETTINGS**: параметры соединения.
- **WINDOW_UPDATE**: управление потоком.

- **RST_STREAM**: прерывание потока.
- **GOAWAY**: graceful shutdown соединения.

Мультиплексирование (Multiplexing)

В HTTP/2 одно TCP-соединение содержит множество **потоков** (streams) — виртуальных каналов, каждый со своим ID. Запросы и ответы чередуются на уровне фреймов:

```
In [ ]: Поток 1: [HEADERS][DATA][DATA]
        Поток 3:      [HEADERS][DATA]
        Поток 1:      [DATA][DATA]
        Поток 5:      [HEADERS]
```

Теперь медленный запрос в потоке 1 **не блокирует** поток 3. HoL на уровне HTTP устранён.

Но! Все потоки разделяют **одно TCP-соединение**. Если один TCP-пакет потерян, TCP требует повторной передачи **всего** соединения. Потоки 1, 3, 5 вынуждены ждать, пока ядро ОС не получит пропущенный пакет. Это **HoL на транспортном уровне** — проблема, которую HTTP/2 не решил.

HPACK: сжатие заголовков

HTTP/1.1 отправляет заголовки в чистом виде при каждом запросе.

HTTP/2 использует **HPACK** — алгоритм сжатия с динамической таблицей. Часто повторяющиеся заголовки (:authority , user-agent , cookie) не отправляются заново, а ссылаются на таблицу.

Server Push (и почему он устарел)

HTTP/2 позволял серверу «проталкивать» ресурсы клиенту до того, как клиент их запросил:

```
In [ ]: Клиент: GET /index.html
        Сервер: 200 OK (index.html)
        Сервер: PUSH_PROMISE /styles.css
        Сервер: PUSH_PROMISE /script.js
```

На практике это оказалось сложным в реализации, приводило к избыточной передаче (клиент мог уже иметь ресурс в кэше) и было **удалено** из HTTP/3. Современная замена — **Early Hints (103)** и приоритизация запросов.

Приоритизация потоков

HTTP/2 позволяет назначать веса потокам. Браузер может сказать: «CSS в потоке 1 — важнее, чем изображение в потоке 3». Сервер (если поддерживает) будет отправлять фреймы CSS чаще.

3.2.3. HTTP/3 и QUIC: переход на UDP

HTTP/3 (2022, RFC 9114) работает поверх **QUIC** — транспортного протокола, построенного поверх **UDP**.

Почему UDP?

TCP — протокол ядра ОС. Изменить TCP невозможно без обновления миллиардов устройств. UDP — минимальный протокол (порт источника, порт назначения, длина, контрольная сумма), всё остальное реализуется в **пользовательском пространстве** (в коде браузера/сервера).

QUIC реализует в пользовательском пространстве:

- Установление соединения с **0-RTT** или **1-RTT** (вместо 1-RTT TCP + 1-RTT TLS = 2-RTT в HTTP/2).
- Надёжную доставку (как TCP), но **на уровне потоков**, а не соединения.
- Шифрование (TLS 1.3) — встроено, не как отдельный слой.
- **Connection Migration**: если клиент меняет IP (переход с Wi-Fi на 4G), соединение не разрывается.

Устранение HoL на транспортном уровне

В QUIC каждый поток имеет **независимую нумерацию пакетов**. Потеря пакета в потоке 1 **не влияет** на поток 3. TCP требует строгого порядка байт в соединении; QUIC — нет.

```
In [ ]: HTTP/2: [TCP Stream] -> HoL при потере любого пакета
        HTTP/3: [QUIC Stream 1] [QUIC Stream 3] [QUIC Stream 5]
                -> потеря в Stream 1 не блокирует Stream 3
```

0-RTT и 1-RTT

- **1-RTT**: первое соединение. QUIC handshake + TLS 1.3 занимают **один** RTT (вместо двух в TCP+TLS).

- **0-RTT**: повторное соединение к известному серверу. Клиент отправляет данные **вместе с первым пакетом**, не дожидаясь подтверждения. Это возможно благодаря кэшированным TLS-параметрам (session resumption).

3.2.4. Почему это важно для FastAPI?

FastAPI работает через **ASGI-сервер** (обычно Uvicorn). Uvicorn использует:

- `h11` — парсер HTTP/1.1.
- `httptools` — более быстрый C-парсер HTTP.
- Для HTTP/2/3 нужны дополнительные библиотеки (`hypercorn` поддерживает HTTP/2 и HTTP/3).

Вывод для архитектора:

- HTTP/1.1 с Keep-Alive достаточен для большинства REST API.
- HTTP/2 критичен, когда клиент делает **много параллельных запросов** к одному серверу (например, gRPC, микросервисы).
- HTTP/3 полезен для **мобильных клиентов** с нестабильным соединением и высоким RTT.

3.2.5. Математическая подоплека: head-of-line blocking как проблема очередей

Рассмотрим систему обслуживания с одним сервером и FIFO-очередью. Время обработки запроса i — S_i . Время ожидания в очереди:

$$W_i = \max(0, W_{i-1} + S_{i-1} - A_i)$$

где A_i — интервал прибытия между запросами $i-1$ и i .

В HTTP/1.1 pipelining очередь строго FIFO: запрос с большим S_i («тяжёлый» ML-inference) задерживает всех последующих. Среднее время ожидания растёт диспропорционально дисперсии S :

$$\mathbb{E}[W] \propto \frac{\lambda \cdot \mathbb{E}[S^2]}{2(1 - \rho)}$$

где λ — интенсивность поступления, $\rho = \lambda \cdot \mathbb{E}[S]$ — загрузка.

HTTP/2 решает это, создавая **несколько виртуальных очередей** (потокaв), обслуживаемых одним сервером (TCP-соединением) с чередованием. Но поскольку TCP — одна очередь с гарантией порядка, потеря пакета восстанавливает проблему на транспортном уровне.

HTTP/3 (QUIC) создаёт **независимые очереди** с независимым восстановлением потерь — идеальное разделение.

3.3. WebSocket

3.3.1. Зачем WebSocket, если есть HTTP?

HTTP — протокол **запрос-ответ**. Клиент всегда инициирует соединение, сервер не может «написать первым». Для real-time приложений (чаты, игры, стриминг метрик ML-модели) это неудобно.

Polling — клиент раз в секунду спрашивает: «есть новые данные?» — создаёт огромную нагрузку.

Long Polling — клиент держит соединение открытым, пока сервер не пришлёт данные. Но после ответа соединение закрывается, и клиент открывает новое.

WebSocket решает проблему радикально: **одно TCP-соединение, полнодуплексный обмен**.

3.3.2. Протокол WebSocket: handshake

WebSocket начинается как обычный HTTP-запрос с заголовками Upgrade:

```
In [ ]: Клиент:
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

Сервер:
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

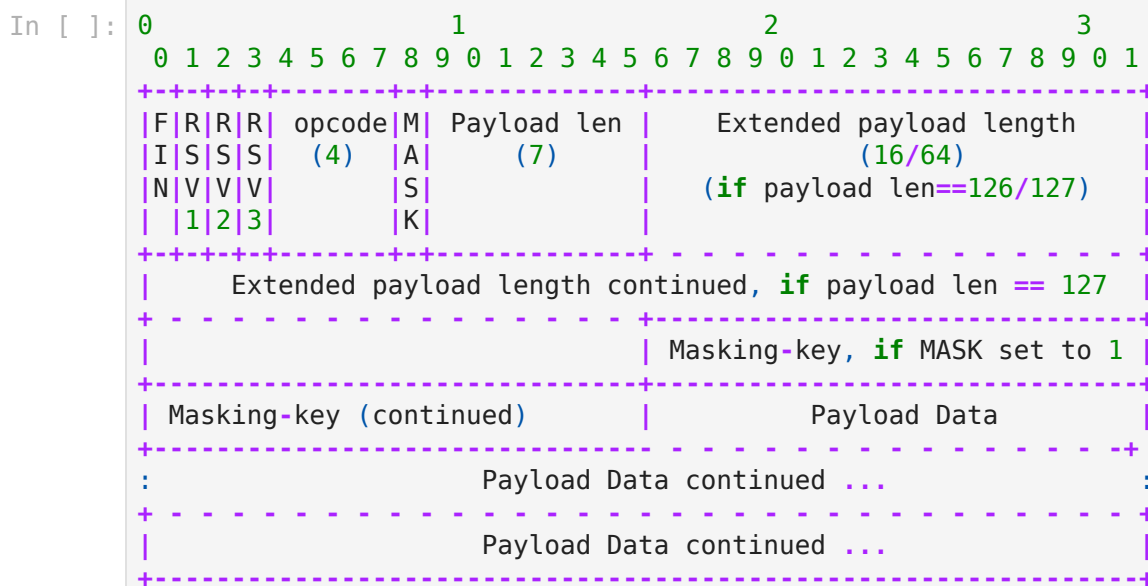
- `Sec-WebSocket-Key` — случайный base64-ключ (16 байт).
- `Sec-WebSocket-Accept` — хеш SHA-1 от ключа + магической строки `258EAF55-E914-47DA-95CA-C5AB0DC85B11`, закодированный в base64.

Это подтверждает, что сервер понимает WebSocket (и не является случайным HTTP-прокси).

После `101` соединение переходит в **бинарный режим WebSocket**. HTTP больше не используется.

3.3.3. Фреймы WebSocket

Данные передаются **фреймами**:



Поля:

- **FIN**: последний фрейм сообщения (сообщение может быть фрагментировано).
- **opcode**: тип фрейма:
 - `0x1` — текстовые данные (UTF-8).
 - `0x2` — бинарные данные.
 - `0x8` — закрытие соединения (close).
 - `0x9` — ping.
 - `0xA` — pong.
- **MASK**: флаг маскирования (всегда 1 для клиент->сервер, 0 для

сервер->клиент).

- **Payload length**: длина данных (7 бит, или 16/64 бита для больших сообщений).

Маскирование (Masking)

Клиент **обязан** маскировать все фреймы с помощью XOR и случайного 32-битного ключа:

$$\text{data}[i] = \text{data}[i] \oplus \text{mask}[i \bmod 4]$$

Зачем? Чтобы предотвратить атаку на кэширующие прокси. Если злоумышленник отправит «вредоносный» HTTP-запрос через WebSocket, маскирование искажает байты, и прокси не сможет интерпретировать их как валидный HTTP.

3.3.4. Ping/Pong и keepalive

WebSocket не имеет встроенного таймаута TCP. Если соединение «молчит», промежуточные прокси (NAT, брандмауэры) могут его разорвать.

Протокол предусматривает **ping/pong**:

- Сервер отправляет **ping** (opcode `0x9`).
- Клиент **обязан** ответить **pong** (opcode `0xA`) с тем же payload.
- Если pong не пришёл вовремя — соединение считается мёртвым.

В FastAPI/Uvicorn ping/pong обычно управляется автоматически.

3.3.5. Когда WebSocket, а когда SSE?

Критерий	WebSocket	SSE (Server-Sent Events)
Направление	Дуплекс (клиент <-> сервер)	Односторонний (сервер -> клиент)
Протокол	TCP с Upgrade	HTTP/1.1 или HTTP/2
Повторное соединение	Ручное	Автоматическое (EventSource API)
Бинарные данные	Нативно	Base64 в тексте
Прокси/брандмауэры	Могут блокировать	Работает через обычный HTTP
Передача cookies	Да	Да

Выбор:

- **SSE** — если сервер только «пушит» данные (логи, прогресс обучения, цены на акции). Проще, работает через HTTP, автоматически переподключается.
- **WebSocket** — если нужен двусторонний обмен (чат, игра, управление роботом, bidirectional streaming аудио для ML-распознавания речи).

3.3.6. Интеграция WebSocket в FastAPI

FastAPI поддерживает WebSocket через Starlette:

```
In [ ]: from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept() # handshake завершён
    try:
        while True:
            data = await websocket.receive_text()
            await websocket.send_text(f"Echo: {data}")
    except Exception:
        await websocket.close()
```

Под капотом:

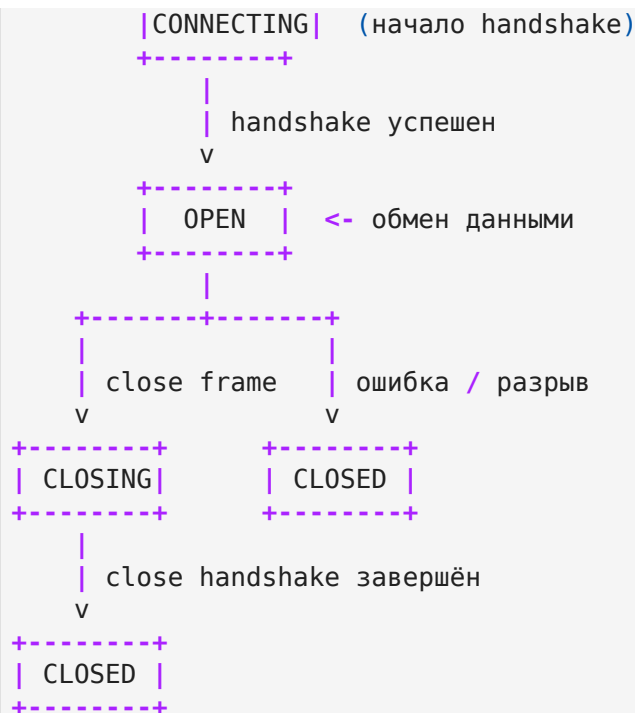
1. FastAPI/Starlette обрабатывает HTTP Upgrade.
2. После `accept()` соединение передаётся в WebSocket-протокол.
3. `receive_text()` / `receive_bytes()` читают фреймы.
4. `send_text()` / `send_bytes()` записывают фреймы.

Важно для ML: WebSocket идеален для streaming-inference, когда клиент отправляет аудио-порции, а сервер возвращает промежуточные результаты распознавания.

3.3.7. Математическая подоплека: конечный автомат WebSocket

Состояния соединения WebSocket:

```
In [ ]: +-----+
```



Формально, каждое сообщение (текст/бинарное) — это событие σ в Σ , переводящее автомат из `OPEN` в `OPEN`. Фрейм `CLOSE` — переход в `CLOSING` или `CLOSED`.

3.4. Асинхронные сетевые библиотеки

3.4.1. `asyncio` низкого уровня: `StreamReader`, `StreamWriter`

Прежде чем использовать высокоуровневые библиотеки, стоит понять, как Python работает с сокетами напрямую.

```
In [ ]: import asyncio

async def tcp_echo_client(message):
    reader, writer = await asyncio.open_connection('127.0.0.1', 8888)

    print(f'Отправляю: {message}')
    writer.write(message.encode())
    await writer.drain() # ждём, пока буфер отправится в ядро

    data = await reader.read(100) # читаем до 100 байт
    print(f'Получил: {data.decode()}')

    writer.close()
    await writer.wait_closed()
```



```
asyncio.run(tcp_echo_client('Hello World'))
```

StreamReader — обёртка над неблокирующим сокетом для чтения. Когда данных нет, `await reader.read()` приостанавливает корутину. Event loop регистрирует сокет в селекторе. Когда данные приходят — корутина возобновляется.

StreamWriter — обёртка для записи. `writer.write()` помещает данные в буфер (не блокируя). `await writer.drain()` ждёт, пока внутренний буфер освободится (backpressure).

```
In [ ]: async def start_server():
        server = await asyncio.start_server(
            handle_client, '127.0.0.1', 8888
        )
        async with server:
            await server.serve_forever()

    async def handle_client(reader, writer):
        addr = writer.get_extra_info('peername')
        print(f"Подключение от {addr}")

        while True:
            data = await reader.read(100)
            if not data:
                break # клиент закрыл соединение
            writer.write(data)
            await writer.drain()

        writer.close()
```

3.4.2. aiohttp: клиент и сервер

aiohttp — зрелая асинхронная библиотека для HTTP.

Клиент:

```
In [ ]: import aiohttp
        import asyncio

    async def fetch():
        # Сессия = пул соединений + куки + общие заголовки
        async with aiohttp.ClientSession() as session:
            async with session.get('https://api.example.com/data') as response:
                print(response.status)
                data = await response.json()
                return data
```

```
asyncio.run(fetch())
```

Ключевые концепции:

- **Session:** хранит пул TCP-соединений (connection pool). Повторное использование соединений избавляет от handshake.
- **Таймауты:** `aiohttp.ClientTimeout(total=10, connect=2)` — общий таймаут 10 секунд, на установление соединения — 2.
- **SSL:** `aiohttp.TCPConnector(ssl=False)` — отключение проверки (только для разработки!).

Connection Pool:

```
In [ ]: connector = aiohttp.TCPConnector(limit=100, limit_per_host=10)
# limit: всего соединений в пуле
# limit_per_host: соединений на один хост
```

3.4.3. httpx : современная альтернатива

`httpx` — библиотека, которая поддерживает **и синхронный, и асинхронный** API, а также HTTP/2.

```
In [ ]: import httpx
import asyncio

async def fetch():
    async with httpx.AsyncClient(http2=True) as client:
        response = await client.get('https://api.example.com/data')
        return response.json()

asyncio.run(fetch())
```

Преимущества `httpx` перед `aiohttp` (клиент):

- Единый API для sync/async.
- Нативная поддержка HTTP/2.
- API ближе к `requests` (привычный интерфейс).
- Лучшая типизация.

Недостатки:

- `aiohttp` быстрее в чистом HTTP/1.1 (написан на C для парсинга).
- `aiohttp` имеет более зрелую серверную часть (хотя FastAPI/Starlette покрывает сервер).

3.4.4. Сравнение библиотек

Библиотека	Асинхронность	HTTP/2	Сервер	Когда использовать
<code>aiohttp</code>	Да	Нет (клиент), частично (сервер)	Да	Микросервисы на чистом <code>aiohttp</code> , сложные клиенты
<code>httpx</code>	Да	Да	Нет	Тестирование FastAPI, HTTP/2-клиенты
<code>urllib3</code>	Нет	Нет	Нет	Синхронный fallback
<code>requests</code>	Нет	Нет	Нет	Синхронные скрипты (не в event loop!)

Золотое правило: никогда не вызывайте синхронный `requests.get()` внутри `async def`. Это заблокирует event loop. Используйте `httpx.AsyncClient` или `aiohttp`.

3.4.5. Математическая подоплека: пулы соединений как система массового обслуживания

Connection pool — это **многоканальная СМО** с конечным числом серверов N (размер пула).

- Если свободное соединение есть — запрос обслуживается немедленно.
- Если все N соединений заняты — запрос ставится в очередь (или получает ошибку/ждёт).

Это модель **M/M/N/K**, где K — максимальный размер очереди.

Вероятность того, что запрос будет обслужен без ожидания (Erlang C formula):

$$P_{\text{wait}} = \frac{\frac{(N\rho)^N}{N!(1-\rho)}}{\frac{(N\rho)^N}{N!(1-\rho)} + \sum_{k=0}^{N-1} \frac{(N\rho)^k}{k!}} + \frac{(N\rho)^N}{N!(1-\rho)}$$

где $\rho = \frac{\lambda}{N\mu}$ — загрузка на один канал.

На практике это означает: если среднее время запроса к внешнему API — 100 мс, а вы делаете 1000 RPS, то пул из 100 соединений будет загружен на $\rho = \frac{1000 \times 0.1}{100} = 1.0$ — система на грани коллапса. Нужно либо увеличить пул, либо уменьшить latency

(кэширование, ближайший дата-центр).

Итог модуля 3

Концепция	Суть	Практическое значение
OSI/TCP/IP	Уровни абстракции сети	Понимание, где живёт ваша проблема (DNS? TCP? HTTP?)
TCP handshake	3 шага для установления соединения	Накладные расходы на каждое новое соединение
TCP AIMD	Экспоненциальный рост, линейное падение окна	TCP «боится» насыщения сети
HTTP/1.1	Текст, Keep-Alive, HoL blocking	Простой, но медленный при множестве запросов
HTTP/2	Бинарные фреймы, мультиплексирование	Устраняет HoL на уровне HTTP, но не TCP
HTTP/3/QUIC	UDP + надёжность в userspace	Устраняет HoL полностью, 0-RTT, миграция соединений
WebSocket	Полный дуплекс поверх TCP	Real-time: чаты, streaming ML-инференс
SSE	Односторонний push поверх HTTP	Прогресс-бары, логи, уведомления
aiohttp/httpx	Асинхронные HTTP-клиенты	Не блокируйте event loop синхронными запросами

В **Модуле 4** мы наконец перейдём к FastAPI: разберём ASGI, жизненный цикл запроса, Pydantic V2 и то, как всё изученное теоретическое материализуется в коде фреймворка.